

Automatización de procesos con n8n para la extracción y gestión de datos públicos: del primer pipeline al diseño arquitectónico del reto final

Autor: Aaron Robles

Universidad Técnica Particular de Loja - Dirección General de Tecnologías de la Información y Transformación Digital

I. RESUMEN / ABSTRACT

Resumen- La Automatización Robótica de Procesos (RPA) permite convertir tareas digitales repetitivas en flujos controlados, verificables y documentados. Este artículo presenta una revisión aplicada de RPA mediante n8n, tomando como base dos momentos de aprendizaje: un primer reto ya implementado y el diseño arquitectónico del reto final orientado al Tablero Macroentorno Ecuador de la Universidad Técnica Particular de Loja. En el primer reto se construyó un pipeline con tres ramas de consulta: Open-Meteo para información climática, REST Countries para indicadores de país y Open Library para recursos académicos; posteriormente, las ramas se integraron en un reporte final estructurado. En el reto final, aún ubicado en la etapa de arquitectura, se propone automatizar la extracción de datos públicos provenientes del Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC. El aporte del trabajo consiste en relacionar los fundamentos de RPA con una solución incremental basada en workflows, integraciones HTTP, normalización, validación, manejo de errores, persistencia en Oracle DB y notificación al equipo de Datos.

Palabras clave- RPA, n8n, automatización, workflows, APIs, Oracle DB, datos públicos, macroentorno Ecuador.

Abstract- Robotic Process Automation (RPA) enables repetitive digital tasks to be transformed into controlled, verifiable, and documented workflows. This paper presents an applied review of RPA using n8n based on two learning stages: a first implemented challenge and the architectural design of the final challenge for the Macroenvironment Dashboard at Universidad Técnica Particular de Loja. The first challenge implemented a three-branch pipeline using Open-Meteo, REST Countries, and Open Library, while the final challenge proposes a scalable architecture for extracting public data from Ecuadorian institutions. The work connects RPA concepts with workflow design, HTTP integrations, data normalization, validation, error handling, Oracle DB persistence, and notifications.

II. INTRODUCCIÓN

Las organizaciones actuales dependen cada vez más de información digital distribuida en plataformas externas, APIs, portales institucionales y servicios web. Aunque muchos procesos se han digitalizado, una parte considerable del trabajo operativo todavía consiste en consultar páginas, descargar archivos, copiar datos, validar resultados, renombrar documentos y comunicar novedades a otros equipos. Estas actividades no siempre requieren razonamiento complejo, pero consumen tiempo, generan dependencia del usuario encargado y aumentan el riesgo de errores por omisión, duplicación o uso de información desactualizada.

En este contexto, la Automatización Robótica de Procesos se presenta como una estrategia orientada a ejecutar tareas repetitivas mediante robots de software. RPA no se limita a imitar acciones humanas sobre una interfaz; también puede integrarse con servicios HTTP, APIs, archivos y bases de datos. Esta evolución permite diseñar flujos más estables que un procedimiento manual, ya que cada ejecución puede registrarse, validarse y repetirse bajo las mismas reglas.

La práctica desarrollada con n8n sigue una cronología progresiva. Primero se construyó un pipeline académico con tres ramas: consulta climática mediante Open-Meteo, consulta de datos de países mediante REST Countries y búsqueda de recursos académicos mediante Open Library. Este primer reto permitió comprender la estructura básica de un workflow: un disparador inicial, nodos de consulta, nodos de transformación, un nodo de unión y un reporte final. A partir de esa experiencia, el reto final amplía el alcance hacia un

escenario institucional: automatizar la obtención de datos públicos para el Tablero Macroentorno Ecuador.

El reto final se encuentra actualmente en la etapa de diseño arquitectónico. Por ello, este artículo no afirma que el pipeline institucional esté completamente implementado, sino que describe la propuesta técnica que guiará su construcción. Esta precisión mantiene coherencia entre el avance real del proyecto y el contenido del documento. La arquitectura propuesta organiza las fuentes por periodicidad, define una capa de extracción, incorpora procesos de normalización y validación, y plantea la persistencia en Oracle DB como repositorio estructurado para el equipo de Datos.

El problema institucional se origina en la necesidad de reunir información publicada por distintas entidades oficiales. Cada fuente posee su propia lógica de publicación: algunas actualizan datos diariamente, otras lo hacen de forma mensual, trimestral o anual, y algunas permanecen como recursos estáticos que se utilizan como base histórica. Si este proceso se realiza manualmente, el responsable debe repetir acciones similares para cada portal, revisar si existe una versión nueva, descargar el recurso y dejar evidencia de que la actualización fue realizada.

El objetivo general del artículo es analizar cómo n8n puede utilizarse como herramienta de RPA para pasar de una práctica de integración de APIs a una arquitectura institucional de extracción de datos. Para cumplir este objetivo, el documento se estructura en ocho apartados: resumen, introducción, marco conceptual, trabajos relacionados, herramientas y tecnologías, tendencias a futuro,

conclusiones y referencias. Esta estructura respeta el formato solicitado y permite presentar la información desde una base teórica hasta la propuesta aplicada.

III. MARCO CONCEPTUAL

A. Automatización Robótica de Procesos

La Automatización Robótica de Procesos se define como el uso de robots de software para ejecutar actividades digitales repetitivas, estructuradas y basadas en reglas. En términos de ingeniería, un robot de software puede entenderse como una secuencia automatizada que recibe entradas, aplica operaciones definidas y genera salidas verificables. Su valor no está únicamente en ahorrar tiempo, sino en convertir una tarea informal en un proceso controlado, medible y documentado.

RPA resulta adecuada cuando el proceso cumple tres condiciones: repetición, estabilidad relativa de reglas y posibilidad de verificación. En el primer reto, estas condiciones aparecen en la consulta repetida de servicios públicos mediante HTTP y en la construcción de un reporte final. En el reto final, se observan en la extracción periódica de archivos de instituciones públicas, la organización por fuente y la necesidad de validar que cada recurso descargado sea correcto antes de entregarlo al equipo de Datos.

Una característica importante de RPA es que no necesariamente reemplaza los sistemas existentes. En muchos casos actúa como una capa de automatización que se ubica por encima de aplicaciones web, APIs, hojas de cálculo o bases de datos. Esta capa ejecuta actividades que anteriormente realizaba un usuario humano, pero lo hace con reglas estables y con la capacidad de registrar cada paso. Por esta razón, RPA es útil cuando la institución no controla directamente las plataformas de origen, como ocurre con los portales públicos utilizados en el reto final.

La automatización puede ser asistida o desatendida. La automatización asistida requiere intervención del usuario durante una parte del proceso, mientras que la automatización desatendida ejecuta tareas de forma programada. El primer reto corresponde principalmente a una ejecución manual controlada, porque se activa desde un nodo Manual Trigger. El reto final, en cambio, se orienta hacia una automatización desatendida mediante Schedule Trigger, debido a que las fuentes deben consultarse según su periodicidad oficial.

B. Workflows, nodos y orquestación

Un workflow representa el recorrido lógico de una automatización. En n8n, este recorrido se modela mediante nodos conectados entre sí. Cada nodo cumple una responsabilidad concreta: iniciar la ejecución, consultar un servicio, transformar datos, dividir lotes, unir ramas, validar condiciones o generar salidas. Esta estructura favorece la modularidad, porque el flujo completo puede dividirse en partes comprensibles y reutilizables.

El primer reto evidencia este enfoque modular. El nodo Inicio de Pipeline activa tres ramas paralelas. La primera rama define ciudades de Ecuador, transforma la lista, itera cada ciudad, consulta Open-Meteo, normaliza temperatura y viento, y calcula estadísticas regionales. La segunda rama

consulta REST Countries y normaliza información de países como nombre, capital, población, área, densidad, moneda e idioma oficial. La tercera rama consulta Open Library y filtra recursos académicos publicados desde 2015. Finalmente, el nodo Unir Ramas de Pipeline integra los resultados y el nodo Reporte Final produce una salida estructurada.

La orquestación consiste en coordinar la ejecución de esas ramas y controlar el orden de las operaciones. En un flujo simple, los nodos se ejecutan de manera lineal. En un flujo más elaborado, pueden existir ramas paralelas, ciclos, validaciones condicionales y rutas de error. El primer reto introduce este principio mediante la división en tres ramas y la posterior consolidación en un reporte. El reto final amplía la idea al organizar fuentes por periodicidad y al proponer esquemas de salida diferenciados.

La modularidad facilita el mantenimiento. Si una API cambia su estructura de respuesta, únicamente se ajusta el nodo de normalización correspondiente. Si una fuente institucional modifica su enlace de descarga, se actualiza el nodo de extracción asociado sin rediseñar todo el pipeline. Esta separación es esencial para que una automatización académica pueda evolucionar hacia una solución institucional mantenible.

C. Trazabilidad, validación y manejo de errores

La trazabilidad es la capacidad de identificar qué acción se ejecutó, cuándo ocurrió, qué fuente fue consultada y cuál fue el resultado obtenido. En automatización, este principio permite auditar ejecuciones y localizar fallos. Para el reto final, la trazabilidad se materializa en la planificación de registros de ejecución, rutas por fuente, esquemas de base de datos y metadatos asociados a cada carga.

La validación es igualmente relevante. Una respuesta HTTP exitosa no garantiza que el archivo o dato recibido sea útil. Puede ocurrir que la fuente devuelva un recurso vacío, una página de error o una estructura distinta a la esperada. Por esta razón, la arquitectura del reto final incorpora una etapa de normalización y verificación antes de cargar datos en Oracle DB. El manejo de errores debe impedir que una falla parcial detenga todo el pipeline y debe generar alertas que faciliten la corrección.

En el primer reto ya se observa una aproximación al control de errores mediante la opción `continueRegularOutput` en nodos HTTP Request y mediante la construcción de objetos de error dentro del nodo de filtrado de recursos académicos. Esta lógica permite que el reporte final pueda marcar un estado incompleto si una rama presenta inconvenientes. El reto final debe fortalecer ese criterio incorporando reintentos, logs y mensajes claros por fuente.

La validación también se relaciona con la calidad de datos. Antes de almacenar información en Oracle DB, el flujo debe revisar si los campos obligatorios existen, si los valores numéricos tienen formato adecuado, si las fechas pueden interpretarse y si la estructura coincide con el esquema esperado. Sin estas reglas, la automatización podría cargar datos incorrectos y trasladar el problema al equipo de Datos.

D. Pipeline de datos y persistencia

Un pipeline de datos es una secuencia de etapas que recibe información desde fuentes externas, la transforma y la entrega en un destino utilizable. En este artículo, el pipeline del primer reto produce un reporte JSON consolidado, mientras que el reto final propone una arquitectura con persistencia en Oracle DB. Esta diferencia marca una evolución importante: de la integración temporal de resultados hacia el almacenamiento estructurado y consultable.

La persistencia en base de datos permite separar la automatización de la visualización final. El workflow no necesita construir directamente el tablero; su responsabilidad es alimentar un repositorio confiable. Posteriormente, el equipo de Datos puede limpiar, consultar, transformar o visualizar la información según sus propios procesos. Esta separación de responsabilidades mejora la arquitectura porque evita concentrar extracción, limpieza, análisis y visualización en un único flujo.

Tabla I. Componentes principales del primer reto realizado en n8n.

Componente	Función dentro del flujo
Inicio de Pipeline	Activa manualmente la ejecución del workflow.
Open-Meteo	Consulta clima por ciudad y calcula estadísticas regionales.
REST Countries	Obtiene indicadores básicos de Ecuador, Colombia y Perú.
Open Library	Filtra recursos académicos sobre automatización.
Merge + Reporte Final	Integra las tres ramas y genera una salida estructurada.

IV. TRABAJOS RELACIONADOS

La literatura sobre RPA coincide en que la automatización aporta valor cuando se aplica a procesos repetitivos, de alto volumen y con reglas claras. Estudios relacionados con automatización empresarial destacan beneficios como reducción de tiempos, disminución de errores y mejora de la consistencia operativa. Sin embargo, también señalan que una automatización mal diseñada puede volverse frágil si depende excesivamente de interfaces cambiantes o si no posee mecanismos de monitoreo y recuperación.

Los trabajos sobre gestión de procesos de negocio y minería de procesos aportan una idea clave para este artículo: antes de automatizar, es necesario comprender el proceso. Esta premisa se refleja en el reto final, donde el primer entregable no es el flujo JSON, sino el diagrama de arquitectura. La arquitectura permite definir fuentes, periodicidades, responsables, herramientas, validaciones y rutas de salida antes de configurar nodos. Así, el diseño previo funciona como una etapa de ingeniería que reduce improvisación.

También existen investigaciones sobre plataformas low-code que resaltan su utilidad para acelerar el desarrollo de soluciones. En el caso de n8n, el modelo basado en nodos permite que un estudiante o equipo técnico construya integraciones sin crear una aplicación completa desde cero. No obstante, el uso de low-code no elimina la necesidad de aplicar buenas prácticas de diseño. El flujo debe tener

nombres claros, separación por responsabilidades, manejo de errores y documentación técnica.

El primer reto se relaciona con trabajos de integración de APIs porque combina servicios externos heterogéneos en un único reporte. El reto final se aproxima más a un pipeline institucional, ya que no solo consulta información, sino que propone una arquitectura para extraer, validar y persistir datos públicos. Esta transición muestra una evolución académica: de una práctica controlada con APIs públicas hacia un diseño de automatización orientado a una necesidad real de la DGTID.

En los estudios de automatización cognitiva se observa una tendencia a incorporar inteligencia artificial, procesamiento de lenguaje natural y análisis contextual. Aunque el alcance actual no requiere modelos de IA para tomar decisiones complejas, estos avances son relevantes porque permiten proyectar mejoras futuras. Por ejemplo, un pipeline podría detectar automáticamente cambios en la estructura de una fuente, clasificar documentos descargados o generar resúmenes de actualización para los usuarios responsables.

Los trabajos sobre gobierno de datos también son pertinentes. Cuando una automatización alimenta un tablero institucional, no basta con que descargue archivos; debe garantizar trazabilidad, control de versiones, validación mínima y responsabilidad sobre los datos cargados. Por ello, el reto final conecta RPA con principios de ingeniería de datos. La automatización se convierte en la primera etapa de una cadena de valor donde los datos pasan de fuentes externas a un repositorio interno preparado para análisis.

Finalmente, las investigaciones sobre arquitectura empresarial destacan la necesidad de diseñar componentes reutilizables y escalables. Esta idea se refleja en la capa de abstracción propuesta para el reto final. En lugar de crear una solución diferente para cada fuente, el pipeline debe aplicar un patrón común: consultar, extraer, normalizar, validar, registrar y cargar. Esta regularidad facilita el mantenimiento y permite incorporar nuevas fuentes en el futuro.

V. HERRAMIENTAS Y TECNOLOGÍAS

A. n8n como plataforma de automatización

n8n es una plataforma de automatización de workflows que permite conectar servicios mediante nodos configurables. En el primer reto se utilizaron nodos de activación manual, nodos HTTP Request, nodos Code, Split in Batches y Merge. El nodo HTTP Request permitió consultar servicios externos; el nodo Code permitió normalizar estructuras JSON y calcular estadísticas; Split in Batches permitió iterar ciudades; y Merge permitió consolidar las ramas del pipeline.

La ventaja técnica de este enfoque es que cada parte del proceso puede probarse de forma aislada. Si falla la consulta climática, se puede corregir esa rama sin modificar la consulta de países o la búsqueda bibliográfica. Esta modularidad será reutilizada en el reto final, donde cada institución pública se concibe como una rama o subworkflow independiente.

El nodo HTTP Request es esencial en automatizaciones basadas en APIs y recursos web. Permite enviar solicitudes a un endpoint, adjuntar parámetros, recibir respuestas JSON,

HTML o archivos, y entregar esos resultados a nodos posteriores. En el primer reto, este nodo se utilizó para consultar Open-Meteo, REST Countries y Open Library. En el reto final, se proyecta su uso para consultar portales y enlaces de descarga asociados a las instituciones públicas.

El nodo Code permite ejecutar lógica personalizada cuando los nodos visuales no son suficientes. En el primer reto se usó para transformar la lista de ciudades, calcular promedios, identificar ciudad más caliente y más fría, normalizar indicadores de país y construir el reporte final. En el reto final, este nodo puede cumplir funciones de validación, estandarización de campos, generación de metadatos, construcción de objetos para Oracle DB y control de errores.

B. APIs públicas utilizadas en el primer reto

Open-Meteo se utilizó como servicio climático porque permite consultar datos a partir de coordenadas geográficas. En el workflow se definieron ciudades de Ecuador con latitud y longitud, luego se iteró cada registro y se obtuvo temperatura y velocidad del viento. REST Countries permitió recuperar información de países mediante códigos, lo que sirvió para normalizar capital, población, área, densidad, moneda e idioma. Open Library permitió consultar recursos bibliográficos relacionados con automatización de procesos y filtrar resultados recientes.

Estas tres integraciones cumplen una función didáctica. Open-Meteo representa una API que requiere parámetros de consulta; REST Countries representa una API que devuelve objetos con estructuras anidadas; y Open Library representa una fuente con resultados múltiples que necesitan filtrado. En conjunto, el reto permitió practicar extracción, transformación, control de errores e integración de resultados.

El flujo exportado del primer reto contiene doce nodos. Esta cantidad es suficiente para demostrar un pipeline completo sin alcanzar la complejidad del reto final. La práctica muestra un patrón reutilizable: definir entradas, consultar fuentes, normalizar datos, calcular indicadores, unir ramas y producir una salida. Ese patrón se conserva conceptualmente en la arquitectura del reto final, aunque las fuentes y el destino cambian.

Un aspecto técnico relevante es que el primer reto no se limita a descargar datos. También calcula estadísticas climáticas, genera densidad poblacional y filtra recursos por año. Esto demuestra que n8n puede usarse no solo como conector, sino como herramienta de procesamiento intermedio. Esta capacidad es importante para el reto final, donde se deberán transformar archivos CSV o Excel antes de persistirlos.

C. Implementación paso a paso del primer reto realizado

El primer reto se implementó como una práctica completa de integración multi-fuente. La solución no se organizó como una secuencia única de nodos, sino como una arquitectura paralela. Esta decisión permitió que cada rama trabajara una fuente diferente y que los resultados se consolidaran únicamente al final. De esta manera, el flujo se mantuvo legible y cada componente pudo probarse de forma independiente.

El primer paso fue crear el nodo Inicio de Pipeline mediante un Manual Trigger. Este nodo funcionó como punto de entrada para activar la ejecución durante las pruebas. Desde este disparador se conectaron tres ramas principales: clima regional, indicadores de país y recursos académicos. Esta separación fue importante porque cada fuente posee una estructura de datos distinta y requiere reglas de transformación específicas.

En la rama de clima regional se creó inicialmente un nodo Set denominado Ciudades de Ecuador. En este nodo se definió una lista con cinco ciudades: Loja, Quito, Guayaquil, Cuenca y Manta. Cada registro incluyó el nombre de la ciudad, su latitud y su longitud. Esta estructura fue necesaria porque la API Open-Meteo requiere coordenadas geográficas para devolver información meteorológica.

Después de definir las ciudades, se utilizó un nodo Code para transformar la lista en elementos individuales. Esta operación convirtió el arreglo de ciudades en registros que n8n podía procesar uno por uno. Posteriormente, el nodo Loop Over Items o Split in Batches permitió iterar sobre cada ciudad y ejecutar una solicitud HTTP independiente por cada registro. Esta parte del flujo demostró cómo n8n puede manejar listas y ciclos dentro de un workflow visual.

El nodo Consultar Clima Open-Meteo utilizó el método GET y parámetros dinámicos para enviar latitud y longitud hacia la API. En lugar de escribir valores fijos, el nodo tomó los datos de cada ciudad durante la iteración. Esta configuración permitió reutilizar el mismo nodo para todas las ciudades. Luego, un nodo Code normalizó la respuesta para conservar únicamente los campos relevantes: ciudad, latitud, longitud, temperatura actual y velocidad del viento.

Una vez procesadas todas las ciudades, se calculó un resumen climático regional. Esta etapa no se limitó a copiar la respuesta de la API, sino que aplicó procesamiento adicional. El flujo calculó temperatura promedio, temperatura máxima, temperatura mínima, ciudad más caliente y ciudad más fría. Con ello, la rama climática produjo información agregada lista para incluirse en el reporte final.

La segunda rama utilizó la API REST Countries para consultar información de Ecuador, Colombia y Perú. El objetivo fue obtener un contexto geográfico y demográfico de países sudamericanos. La respuesta de esta API contiene objetos anidados, por lo que fue necesario aplicar normalización mediante JavaScript. El nodo Code extrajo nombre, capital, población, área territorial, moneda e idioma oficial. Además, calculó la densidad poblacional dividiendo población por área.

La tercera rama utilizó Open Library para buscar recursos académicos relacionados con automatización de procesos. Esta rama tuvo dos propósitos. Primero, permitió integrar una fuente bibliográfica distinta a las fuentes climáticas y geográficas. Segundo, sirvió para implementar manejo de errores. Si la API devolvía una respuesta inválida o no entregaba el arreglo esperado de documentos, el nodo Code construía un objeto de error con nodo afectado, fuente, mensaje y timestamp.

Finalmente, las tres ramas se conectaron al nodo Unir Ramas de Pipeline. Este nodo Merge recibió las salidas de clima, país y recursos académicos. Después, el nodo Reporte Final integró la información en un único objeto JSON. El reporte incluyó versión, timestamp, estado, fuentes utilizadas, clima regional, contexto de país, recursos académicos y arreglo de errores. Si alguna rama generaba un error, el estado cambiaba de completo a incompleto.

El resultado del primer reto fue un workflow funcional que demostró las capacidades esenciales de n8n: activación manual, consumo de APIs, uso de parámetros dinámicos, iteración, transformación con JavaScript, consolidación de ramas, manejo parcial de errores y generación de una salida estructurada. Estas capacidades constituyen la base técnica que se reutiliza en el diseño del reto final.

Tabla II. Secuencia técnica del primer reto realizado en n8n.

Etap	Nodo principal	Resultado esperado
Activación	Manual Trigger	Inicio controlado del pipeline.
Entrada climática	Set + Code	Lista de ciudades convertida en elementos.
Iteración	Loop Over Items	Procesamiento individual por ciudad.
Consulta API	HTTP Request	Respuesta de Open-Meteo, REST Countries u Open Library.
Transformación	Code	Datos normalizados y cálculos derivados.
Consolidación	Merge	Unión de las tres ramas del workflow.
Salida	Reporte Final	JSON estructurado con estado, fuentes y errores.

D. Análisis del flujo y decisiones de diseño del primer reto

La decisión de usar arquitectura paralela tuvo una justificación técnica. Si todas las consultas se hubieran colocado en una sola línea de ejecución, el workflow sería más difícil de leer y mantener. Al separar las ramas, cada fuente conserva su propia lógica y se reduce el acoplamiento entre componentes. Esta separación también permite identificar rápidamente qué rama falla durante una prueba.

El uso de JavaScript en nodos Code respondió a la necesidad de transformar estructuras JSON heterogéneas. Open-Meteo, REST Countries y Open Library no devuelven datos con la misma forma. Por ello, cada respuesta debía adaptarse a una estructura común antes de integrarse. La normalización es una práctica fundamental en pipelines de datos, porque permite que el resultado final sea predecible y reutilizable.

El manejo de errores se diseñó bajo el criterio de tolerancia a fallos parciales. En los nodos HTTP Request se configuró la opción On Error: Continue Regular Output. Con esta configuración, si una API fallaba, el pipeline no se detenía por completo. En su lugar, el error podía pasar a la etapa de consolidación. Esta práctica es importante porque en integraciones reales las fuentes externas pueden fallar temporalmente.

El reporte final fue diseñado como un objeto JSON reutilizable. Su estructura centraliza el resultado de todas las ramas y permite que otro sistema lo consuma posteriormente. Aunque en el primer reto no se almacenó en una base de datos, la forma del reporte anticipa una arquitectura más avanzada. En el reto final, esta idea evoluciona hacia una salida persistente en Oracle DB.

La práctica también permitió evidenciar la importancia de nombrar correctamente los nodos. Nombres como Normalizar Datos Climáticos, Filtrar Recursos Académicos o Calcular Estadísticas Climáticas explican la finalidad del componente sin necesidad de abrir su configuración interna. Esta convención de nombres será necesaria en el reto final, donde el número de fuentes y validaciones será mayor.

El primer reto dejó una lección central: automatizar no consiste únicamente en conectar APIs. También implica definir entradas, transformar respuestas, controlar errores, diseñar una salida comprensible y documentar el flujo. Por esta razón, la experiencia sirve como antecedente directo para el reto final, en el cual la complejidad aumenta por la variedad de fuentes públicas, periodicidades y necesidad de persistencia.



Fig. 1. Workflow del primer reto implementado en n8n: integración de Open-Meteo, REST Countries y Open Library.

E. Relación entre el primer reto y el reto final

El primer reto y el reto final no deben analizarse como prácticas aisladas. El primero cumple la función de base técnica porque demuestra cómo construir un pipeline multi-fuente en n8n. El segundo toma esa base y la proyecta hacia un caso institucional más exigente. La diferencia principal está en el alcance: el primer reto produce un reporte JSON académico, mientras que el reto final debe diseñar una arquitectura para obtener datos públicos, validarlos, registrar errores, cargarlos en Oracle DB y notificar al equipo de Datos.

La rama de Open-Meteo se relaciona con las fuentes diarias del reto final, como petróleo WTI o riesgo país, porque ambas requieren consultas frecuentes y control de fecha. La rama de REST Countries se relaciona con la normalización de estructuras, porque enseña a extraer campos útiles desde respuestas complejas. La rama de Open Library se relaciona con el control de errores y filtrado de resultados, aspectos que serán necesarios cuando alguna fuente pública cambie su formato o no responda.

El nodo Merge del primer reto anticipa la necesidad de consolidación del reto final. En la arquitectura institucional, las fuentes no necesariamente se unirán en un solo JSON, pero sí deberán reportar un estado común de ejecución. Por ello, se plantea una capa de registro que indique qué fuentes se procesaron correctamente, cuáles fallaron y qué acciones se tomaron.

El primer reto también justifica la capa de abstracción propuesta para el reto final. En ambos casos se repiten operaciones comunes: consulta, transformación, validación y salida. La diferencia es que el reto final requiere que estas operaciones sean más robustas, porque el resultado debe alimentar un tablero institucional. Por tanto, la experiencia inicial permite comprender el patrón antes de aplicarlo a fuentes de mayor responsabilidad.

F. Estructura del reporte final del primer reto

La estructura del reporte final fue un elemento clave porque permitió integrar salidas heterogéneas dentro de un mismo objeto. En lugar de entregar tres respuestas separadas, el nodo final construyó un contenedor común denominado reporte. Esta decisión facilita que el resultado sea leído por otro proceso, almacenado posteriormente o enviado como evidencia de ejecución.

El reporte incluyó campos de control y campos de contenido. Los campos de control fueron version, timestamp, estado, fuentes y errores. Estos campos permiten saber cuándo se ejecutó el flujo, qué fuentes participaron y si el procesamiento fue completo o incompleto. Los campos de contenido fueron clima_regional, contexto_pais y recursos_academicos. Esta separación hace que el JSON sea más claro y fácil de validar.

La propiedad estado se diseñó como un indicador global de ejecución. Cuando todas las ramas entregaban resultados válidos, el valor permanecía como completo. Si alguna rama enviaba un objeto de error, el nodo final cambiaba el estado a incompleto y agregaba el detalle dentro del arreglo errores. Esta lógica permite comunicar fallos sin perder los datos que sí fueron procesados correctamente.

La estructura final también demuestra un principio que debe trasladarse al reto final: todo pipeline debe producir una salida interpretable por humanos y sistemas. En el reto institucional, esa salida puede tomar la forma de registros en Oracle DB, archivos con metadatos y notificaciones. Sin embargo, el criterio es el mismo: cada ejecución debe resumir qué ocurrió y permitir tomar decisiones posteriores.

Por esta razón, el primer reto funciona como una prueba de concepto de observabilidad básica. Aunque no incorpora un tablero de monitoreo, sí genera evidencia mínima de ejecución. En fases posteriores, esos campos podrían

almacenarse como logs, permitiendo construir estadísticas sobre disponibilidad de fuentes, tiempos de respuesta y frecuencia de errores.

Tabla III. Campos principales del reporte JSON generado en el primer reto.

Campo	Propósito
version	Identificar la versión lógica del pipeline.
timestamp	Registrar fecha y hora de ejecución.
estado	Indicar si la ejecución fue completa o incompleta.
fuentes	Enumerar las APIs consultadas.
clima_regional	Guardar estadísticas climáticas agregadas.
contexto_pais	Guardar indicadores normalizados de países.
recursos_academicos	Guardar libros filtrados desde Open Library.
errores	Registrar fallos detectados por rama.

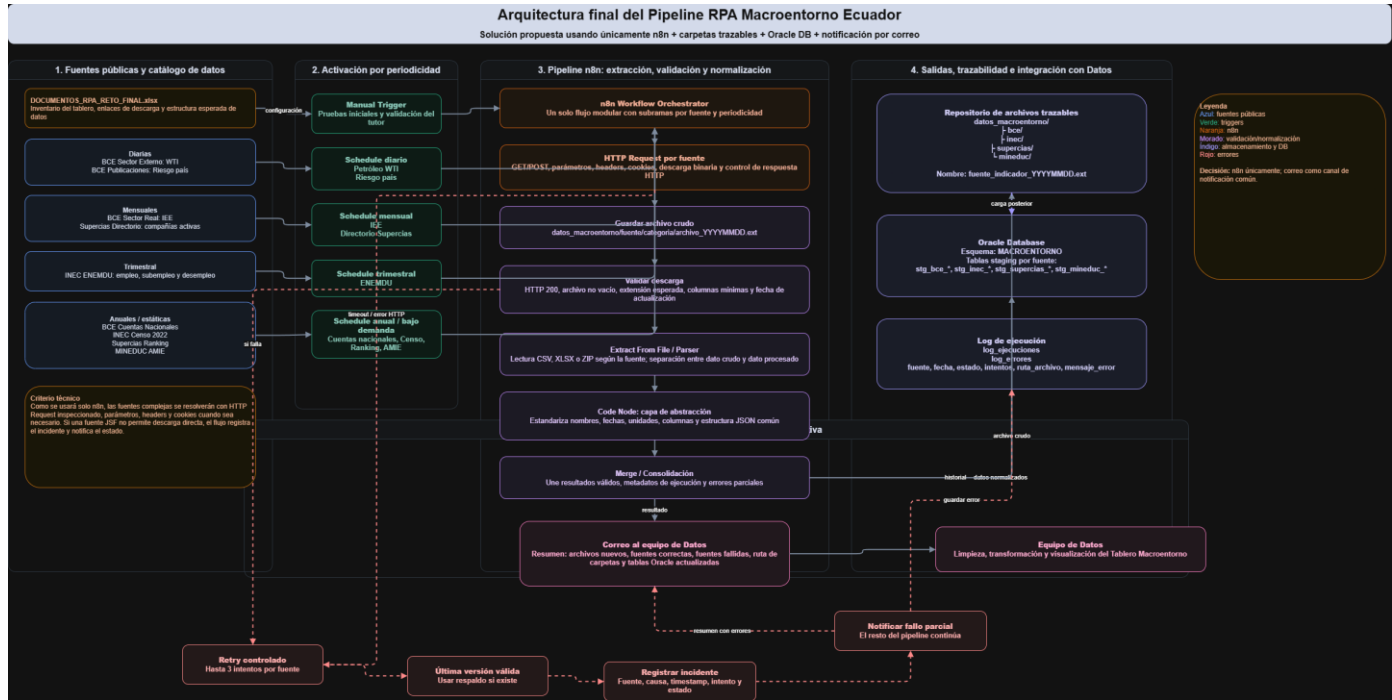


Fig. 2. Arquitectura resumida del reto final propuesta a partir del diagrama en draw.io.

G. Arquitectura propuesta del reto final

El reto final plantea automatizar fuentes públicas del macroentorno ecuatoriano. A diferencia del primer reto, el objetivo no es producir un reporte académico simple, sino diseñar un pipeline que sirva como insumo para el equipo de Datos. La arquitectura se organiza por periodicidad: flujos diarios para petróleo y riesgo país; flujos mensuales para sector real y directorio de compañías; flujos trimestrales para ENEMDU; y flujos anuales o estáticos para cuentas nacionales, ranking de empresas, registros administrativos de MINEDUC y censo.

La arquitectura diseñada en draw.io incorpora una secuencia común: Schedule Trigger, HTTP Request, Extract From File, Code para estructurar información y Oracle DB como destino. Esta secuencia representa una capa de abstracción porque separa la fuente original de la forma final en que los datos serán almacenados. Aunque cada fuente tenga formato diferente, el pipeline debe convertirla en una estructura controlada antes de persistirla.

El uso de Oracle DB como nodo final responde a la necesidad de una persistencia más formal que una simple carpeta local. Una base de datos permite almacenar registros normalizados, controlar esquemas por fuente, facilitar

consultas posteriores y servir de punto común para el equipo de Datos. No obstante, el diseño también debe conservar evidencia de archivo original o metadatos de descarga, porque la trazabilidad sigue siendo un requisito del reto.

El diseño por periodicidad ayuda a evitar ejecuciones innecesarias. No tendría sentido consultar todos los recursos diariamente si algunos se publican de forma anual o trimestral. Por ello, la arquitectura separa flujos diarios, mensuales, trimestrales y anuales. Esta decisión reduce carga operativa, permite programar cada rama con una frecuencia adecuada y facilita la revisión del historial de ejecuciones.

El diagrama de arquitectura también funciona como herramienta de comunicación. Antes de implementar los nodos, el tutor y el estudiante pueden revisar el alcance, detectar fuentes demasiado complejas, identificar dependencias y acordar criterios de evaluación. Esta función es importante porque evita avanzar hacia una solución técnicamente equivocada o demasiado ambiciosa para el tiempo disponible.

Tabla IV. Organización técnica propuesta para el reto final.

Periodicidad	Fuentes	Salida propuesta
Diaria	BCE petróleo WTI y riesgo país	Tablas Oracle + log de ejecución
Mensual	BCE sector real y Supercias directorio	Esquema por fuente
Trimestral	INEC ENEMDU	Indicadores laborales normalizados
Anual/estática	BCE cuentas, Supercias ranking, MINEDUC y Censo	Datos históricos y registros base

H. Capa de abstracción, validación y Oracle DB

La capa de abstracción del pipeline evita que el resto del sistema dependa directamente de los formatos originales. Esta capa se ubica entre la extracción y la persistencia. Su responsabilidad es limpiar nombres de campos, seleccionar hojas relevantes, convertir tipos de datos, agregar metadatos de fecha y preparar registros para Oracle DB. De esta forma, el equipo de Datos no tendría que interpretar cada fuente desde cero, sino consumir estructuras ya organizadas.

En términos prácticos, esta capa puede implementarse mediante nodos Code, expresiones de n8n o subworkflows reutilizables. Por ejemplo, una regla común puede validar que la respuesta no esté vacía, otra puede construir el nombre lógico de la fuente, otra puede agregar fecha de descarga y otra puede decidir si se carga la información o se registra un error. Este enfoque reduce duplicación y facilita mantenimiento cuando se agreguen nuevas fuentes.

La persistencia en Oracle DB exige pensar en esquemas. Cada fuente puede almacenarse en una tabla específica o en un conjunto de tablas relacionadas, dependiendo de su granularidad. Los indicadores diarios pueden requerir claves por fecha; los registros anuales pueden requerir campos de período; y los datos estáticos pueden requerir control de versión. Estas decisiones deben documentarse porque afectan la forma en que el equipo de Datos consultará la información.

La capa de abstracción también permite aplicar reglas de seguridad. Las credenciales de base de datos no deben escribirse directamente en el código ni compartirse dentro de documentos públicos. En una implementación real, deben almacenarse en credenciales seguras de n8n o en un gestor de secretos. Esta consideración es parte del paso de una práctica académica a una arquitectura institucional.

I. Criterios de validación del reto final

Para evaluar el pipeline del reto final se deben aplicar criterios objetivos. El primero es la disponibilidad de la fuente: el flujo debe verificar si la respuesta existe y si el servidor no devolvió error. El segundo es la integridad: el archivo o respuesta no debe estar vacío. El tercero es el formato: se debe confirmar si el recurso corresponde al tipo esperado, por ejemplo CSV, Excel o HTML parseable.

El cuarto criterio es la coherencia estructural. Si el archivo es una hoja de cálculo, deben existir hojas o columnas esperadas; si la fuente devuelve JSON, deben existir campos obligatorios. El quinto criterio es la carga en Oracle DB: la operación debe registrar cuántos registros fueron insertados o actualizados. Finalmente, el sexto criterio es la notificación: el equipo de Datos debe recibir un resumen claro de fuentes exitosas, fuentes fallidas y mensajes de error.

Tabla V. Criterios de control recomendados para la validación del pipeline.

Criterio	Pregunta de control
Disponibilidad	¿La fuente respondió correctamente?
Integridad	¿El archivo o respuesta contiene datos?
Formato	¿La extensión o estructura es la esperada?
Coherencia	¿Existen campos u hojas obligatorias?
Persistencia	¿Los datos se cargaron en Oracle DB?
Notificación	¿Se informó el resultado al equipo de Datos?

J. Comparación técnica entre el primer reto y el reto final

La comparación entre ambos retos permite observar una progresión de complejidad. En el primer reto las fuentes son APIs públicas con respuestas relativamente estables y bien delimitadas. En el reto final, en cambio, se trabaja con portales institucionales que pueden publicar archivos en formatos diferentes, enlaces variables o páginas que requieren inspección previa. Esta diferencia obliga a diseñar un pipeline más flexible.

En el primer reto la salida es un JSON consolidado. En el reto final la salida debe convertirse en datos persistentes dentro de Oracle DB y, posiblemente, en archivos organizados con trazabilidad. Esto cambia la naturaleza de la solución: ya no basta con mostrar un resultado, sino que se debe garantizar que el equipo de Datos pueda reutilizarlo en procesos posteriores.

El manejo de errores también evoluciona. En el primer reto se probó una estrategia básica para continuar la ejecución aunque una API falle. En el reto final se requiere una estrategia más completa: reintentos, registro por fuente, mensajes comprensibles, uso de última versión válida si corresponde y notificación de incidencias. Esta evolución responde al carácter institucional del sistema.

La trazabilidad pasa de ser una característica deseable a convertirse en un requisito. En el primer reto, el timestamp del reporte indica cuándo se generó la salida. En el reto final, cada fuente debe conservar evidencia de fecha, versión, origen, estado de validación y resultado de carga. Esta información permite auditar el pipeline y reproducir análisis históricos.

Finalmente, el primer reto se ejecuta de forma manual; el reto final debe avanzar hacia ejecución programada. Esta diferencia introduce requisitos de disponibilidad, monitoreo y mantenimiento. Un flujo manual puede depender del estudiante; un flujo programado debe funcionar con autonomía y reportar fallos cuando el usuario no está observando directamente la ejecución.

Tabla VI. Evolución técnica desde el primer reto hacia el reto final.

Aspecto	Primer reto	Reto final
Fuentes	APIs públicas: Open-Meteo, REST Countries y Open Library.	Instituciones públicas: BCE, INEC, Supercias y MINEDUC.
Activación	Manual Trigger.	Schedule Trigger por periodicidad.
Salida	Reporte JSON consolidado.	Carga en Oracle DB, trazabilidad y notificación.
Errores	Continuar salida regular y registrar error básico.	Reintentos, logs, alerta y control por fuente.
Alcance	Práctica académica multi-fuente.	Pipeline institucional para equipo de Datos.

K. Recomendaciones para la implementación posterior

Para la siguiente fase se recomienda implementar primero una fuente de baja complejidad, como una descarga directa en CSV o Excel. Esta estrategia permite validar la arquitectura sin enfrentar de inmediato los casos más difíciles. Una vez probado el patrón básico, se puede reutilizar para las demás fuentes.

También se recomienda construir subworkflows reutilizables. Un subworkflow puede encargarse de registrar errores; otro, de validar estructura; otro, de preparar metadatos; y otro, de cargar en Oracle DB. Esta separación evita repetir la misma lógica en todas las fuentes y facilita realizar cambios globales.

La carga en Oracle DB debe manejar idempotencia. Si el pipeline se ejecuta dos veces el mismo día, no debería duplicar registros. Para ello se pueden usar claves compuestas por fuente, fecha y periodo, o estrategias de actualización controlada. Este criterio es fundamental para mantener la calidad del repositorio.

Finalmente, la documentación debe mantenerse junto al flujo. Cada nodo debe tener nombre descriptivo y, cuando sea necesario, notas internas que expliquen su función. Además, el informe final debe registrar decisiones, dificultades, fuentes cubiertas, errores encontrados y evidencia de ejecución automática. Esta documentación facilitará la evaluación y el mantenimiento del sistema.

VI. TENDENCIAS A FUTURO

La automatización de procesos evoluciona hacia modelos más inteligentes, observables y conectados con infraestructura cloud. En el contexto de n8n, una tendencia relevante es la combinación de workflows con capacidades de inteligencia artificial para clasificar documentos, interpretar estructuras variables y asistir en la toma de decisiones operativas. Aunque el reto actual se centra en extracción y persistencia, en fases futuras se podría incorporar análisis automático de cambios entre versiones, detección de anomalías y generación de reportes ejecutivos.

Otra tendencia importante es la hiperautomatización, entendida como la integración de RPA, APIs, analítica, minería de procesos y sistemas de datos. Bajo este enfoque, la automatización no solo ejecuta tareas, sino que también identifica oportunidades de mejora y produce evidencia para optimizar procesos. Para el reto final, esto implicaría pasar de un pipeline de descarga a una plataforma que supervise

fuentes, mida tiempos de ejecución, detecte fallos recurrentes y sugiera ajustes.

La observabilidad será cada vez más necesaria. Un pipeline institucional no debe depender únicamente de que el usuario revise si el flujo se ejecutó. Debe contar con logs, métricas, estados, alertas y tableros de monitoreo. En una versión futura, cada ejecución podría registrar duración, número de registros cargados, estado de validación, fuente afectada y mensaje de error. Esta información permitiría evaluar estabilidad y calidad operativa del sistema.

También se proyecta una mayor integración con bases de datos y servicios empresariales. La persistencia en Oracle DB puede convertirse en el punto de partida para procesos ETL, tableros de visualización, consultas históricas y control de versiones. Además, el flujo podría desplegarse en un servidor institucional para evitar dependencia de una computadora local y garantizar disponibilidad. En ese escenario, la seguridad de credenciales, el control de accesos y los respaldos serían componentes obligatorios.

Finalmente, la experiencia del primer reto demuestra que los workflows pueden comenzar como ejercicios académicos y evolucionar hacia soluciones institucionales. La tendencia más relevante no es únicamente usar más herramientas, sino diseñar automatizaciones mantenibles, documentadas y alineadas con necesidades reales. Esta visión convierte a n8n en una herramienta formativa y productiva dentro de la ingeniería de procesos digitales.

Una mejora futura específica consiste en implementar comparación entre versiones. El pipeline podría descargar un archivo, calcular una huella digital o hash, compararlo con la versión anterior y decidir si realmente debe cargarse de nuevo. Esto evitaría reprocesar archivos idénticos y permitiría notificar únicamente cuando existan cambios relevantes. Esta práctica sería útil en fuentes anuales o estáticas.

Otra proyección es separar el flujo en subworkflows reutilizables. Un subworkflow puede encargarse de registrar errores, otro de validar archivos, otro de preparar metadatos y otro de cargar en Oracle DB. Con esta estrategia, las nuevas fuentes no se construirían desde cero, sino que reutilizarían componentes comunes. Esto reduce errores de implementación y facilita pruebas.

La arquitectura también puede ampliarse con un tablero de monitoreo. Este tablero podría mostrar la última ejecución por fuente, su estado, el tiempo de respuesta, la cantidad de registros cargados y el detalle de errores. Así, la automatización dejaría de ser un proceso oculto dentro de n8n y se convertiría en un servicio observable para usuarios técnicos y no técnicos.

VII. CONCLUSIONES

El artículo presentó una línea cronológica de aprendizaje en automatización con n8n. El primer reto permitió construir un pipeline funcional con tres ramas de integración: clima, contexto de país y recursos académicos. Esta práctica permitió comprender el uso de disparadores, solicitudes HTTP, transformación mediante código, unión de ramas y generación de un reporte final.

El reto final amplía esa experiencia hacia un problema institucional de mayor alcance: la automatización de fuentes públicas del macroentorno ecuatoriano. Debido a que el proyecto aún se encuentra en la etapa de arquitectura, el aporte principal actual es el diseño del pipeline, no la ejecución completa de todas las fuentes. Esta arquitectura permite ordenar el trabajo por periodicidad, separar responsabilidades y preparar la implementación posterior.

La incorporación de Oracle DB como destino fortalece la propuesta porque convierte la salida del pipeline en una persistencia estructurada. Sin embargo, esta decisión exige una capa de abstracción que normalice datos antes de cargarlos y que mantenga trazabilidad de la fuente original. La arquitectura debe contemplar validación, manejo de errores, logs y notificación para que el flujo sea útil más allá de una demostración puntual.

n8n resulta adecuado para este tipo de automatización porque combina una interfaz visual con la posibilidad de ejecutar lógica personalizada. Su modelo basado en nodos facilita la explicación académica y permite construir soluciones modulares. No obstante, la herramienta debe usarse con criterios de ingeniería: nombres descriptivos, documentación, separación por fuente, pruebas individuales y control de errores.

En síntesis, el trabajo demuestra que RPA no debe entenderse solo como la acción de automatizar clics o consultas, sino como el diseño de procesos digitales verificables. La evolución desde el primer reto hasta la arquitectura del reto final evidencia una progresión técnica coherente: de la integración básica de APIs hacia un pipeline institucional orientado a datos públicos, persistencia y trazabilidad.

Como trabajo pendiente, la siguiente fase será implementar los flujos definidos en la arquitectura, probar cada fuente, configurar los disparadores periódicos, validar la carga en Oracle DB y documentar el historial de ejecuciones. Esta continuidad permitirá convertir el diseño actual en un pipeline operativo y evaluable según los criterios del reto final.

VIII. REFERENCIAS

- [1] L. Willcocks y M. Lacity, "Enterprise Automation and Digital Transformation in Telecommunications", MIS Quarterly Executive, 2016.
- [2] W. M. P. van der Aalst, "Business Process Automation and Process Intelligence", Business & Information Systems Engineering, 2018.
- [3] R. Syed et al., "Robotic Process Automation: Contemporary themes and challenges", Computers in Industry, vol. 115, 2020.
- [4] M. Dumas, M. La Rosa, J. Mendling y H. A. Reijers, Fundamentals of Business Process Management. Springer, 2018.
- [5] A. Asatiani y E. Penttinen, "Turning robotic process automation into commercial success", Journal of Information Technology Teaching Cases, 2016.
- [6] n8n, "HTTP Request node documentation", n8n Docs, 2026.
- [7] n8n, "Code node documentation", n8n Docs, 2026.
- [8] Open-Meteo, "Weather Forecast API Documentation", Open-Meteo, 2026.
- [9] REST Countries, "REST Countries API", 2026.
- [10] Open Library, "Search API Documentation", Internet Archive, 2025.

- [11] DGTITD-UTPL, "Pipeline de extracción automatizada de datos públicos del macroentorno ecuatoriano: reto de 7 semanas", Universidad Técnica Particular de Loja, 2026.
- [12] A. Robles, "Primer Reto: workflow de integración con Open-Meteo, REST Countries y Open Library", archivo JSON de n8n, 2026.